

Alexey Yurchenko

AI Agent Engineer / Senior Fullstack Engineer

Email: alexes.dev@gmail.com

GitHub: github.com/alexesdev

Remote / Digital nomad

ABOUT ME

I'm looking for an **AI Agent Developer** role. These days I write most of my code for agents that help people.

15 years as a developer since 2011 — no full break, not even through the 2024 burnout, when writing code felt awful and I still couldn't stop. AI tools pulled me out of that pit: now I can put all my experience to work while the LLM types code 5x faster than a human can. Right now I run iiminion.ru, where I package autonomous AI agents and sell them by subscription: an agent that watches competitors and writes posts in a brand voice, a script-to-video pipeline, a landing-page generator that survives client edits, plus per-customer fine-tuning. I also built the infrastructure underneath — an agent-orchestration platform (Hermes agents on a Nomad cluster, coordinating through shared memory).

Go and TypeScript are my main languages. Most of my career has been about taking unclear specs and turning them into something that runs. In 2025 I changed how I work: small, steady steps toward goals that keep shifting under the market.

FEATURED PROJECTS

trip2g.com — Founder, Sole Engineer (open source, MIT)

2025 — Present

A federated protocol that turns markdown note collections (Obsidian, Notion, Google Drive, Telegram) into RAG sources for AI agents over Model Context Protocol. About 12 months of solo work from the first prototype to a public hub where other people now plug in their own bases.

It didn't start as an AI project. I'd been keeping an Obsidian vault for three years and wanted a way to share parts of it. Writing long posts is hard; writing 5–10 short interlinked notes is much easier. The original plan was a knowledge-graph engine where you could carve out a subgraph and sell it. I knew AI should fit in somewhere but couldn't see how. Once I tried MCP, the picture clicked: the base becomes a private external RAG, the sensitive data stays on the user's machine, and the agent — not the human — is the natural consumer. Everything below is what I had to build to make that workable. The same realization later spun off both iiminion.ru and the AI Agent Orchestration Platform.

- Markdown-first knowledge engine — ingests markdown, renders each note as a page, exposes the corpus as a typed knowledge graph; arbitrary schemas and monetizable subgraphs can be built on top.
- MCP server as external RAG — keeps sensitive data local; the agent issues targeted queries instead of shipping the whole corpus to a third-party service.
- Self-describing bases — each base ships its own search instructions, so the calling agent plans its own retrieval.
- Vector "router" base — a meta-base over other bases; semantic search routes a query to the right downstream sources, then the agent fans out across the matches.
- Protocol-level federation with adapters for Notion, Google Drive, Telegram, etc. — direct hub-to-hub peer-to-peer, no central relay.
- Template engine with full access to the markdown AST — extract third-level sections as cards, combine several notes into a landing page, etc. Agents update markdown and the rendered site updates with it.

- Recent features: per-heading note retrieval and TOC over MCP; admin-mode API key so an agent can manage the system itself; BEM templating that makes agent-driven landing-page edits structurally stable.
- External users are building their own subgraphs and case-study bases on the protocol.

Built in Go (single binary, deploys anywhere) on SQLite + Litestream. SQLite gives the app stateful local speed; Litestream streams every write out to S3, which turns the persistence story effectively stateless from the orchestrator's point of view — Nomad can move the container between nodes and lose nothing, S3 carries the durability. When I started this, there weren't good databases that sat directly on top of S3 yet, so the hybrid (hot SQLite cache on the server, replicated state on S3) was the cheapest reliable path. 15 years of small infrastructure choices baked in.

simplecloud.2pub.me — hosting platform for trip2g instances. The panel manages a Nomad cluster; adding a server pre-provisions warm container slots, so a new user gets an instant, already-running instance instead of waiting through a cold start. The orchestration platform below now deploys Hermes agents through the same panel.

AI Agent Orchestration Platform — Independent R&D

2025 — Present

Spun off from a trip2g realization: if a private knowledge base is what an agent needs as memory, every agent should get one of its own.

- Each Hermes agent gets its own dedicated trip2g instance as long-term memory — every action recorded with rationale, the entire knowledge browsable through trip2g's wiki-style web UI, full audit trail.
- Multi-agent coordination through shared memory: agents read each other's instances to align on context, hand off tasks, avoid duplicate work — a lightweight alternative to heavy orchestration frameworks.
- Deployment runs through the simplecloud panel above — same Nomad cluster, same warm-slot provisioning, same instant rollout.
- Currently exploring agent evolution: when the base skills layer or the underlying engine changes, a new agent reads the second brain of its ancestor and inherits the context. Groundwork for automated evolutionary testing, a multi-month effort.

WORK EXPERIENCE

Founder & AI Engineer — iiminion.ru

2026 — Present | Remote

Productized autonomous AI agents on top of my open-source stack (trip2g + Hermes orchestration), sold by subscription. The subscriptions help fund further open-source work. Agents are packaged for self-serve and fine-tuned per customer.

- **Competitor-monitoring agent:** tracks competitors and produces branded social posts and content for the customer's product in their voice — research, drafting, and publishing.
- **Script-to-video pipeline:** end-to-end automated short-form video generation from a written script.
- **Edit-stable landing-page generator:** produces marketing landing pages that survive iterative client edits without structural breakage.

Backend Developer, Tool Team — P2P.org (P2P Validator)

08/2022 — 01/2024 | Remote

Infrastructure and data platform for Proof-of-Stake blockchain networks. The best engineering culture I've worked in.

- Built data extractors in **Python and Go** for Cosmos SDK-based networks; pipelines processed terabytes of on-chain data into BigQuery for the analytics team.
- Shipped gRPC + Protobuf APIs for the analytics team to consume pipeline outputs.

- Built an open-source [viewer for Agoric's on-chain value tree](#) in React / TypeScript while running the Agoric integration at P2P solo. Started it as a way to figure out the chain for myself; the Agoric core team picked it up and used it daily, then rewrote it from scratch on the same concept — now hosted at [vstorage.agoric.net](#).
- Owned the data pipelines end to end: chain extraction → transformations → BigQuery tables the analysts queried.
- Deployed on Google Cloud / Kubernetes via Docker; collaborated cross-team on schema design and data SLAs.

Stack and tools: Python, Go, React, TypeScript, BigQuery, gRPC / Protobuf, Docker, Kubernetes, Google Cloud, Cosmos SDK, Cosmos Hub, Agoric.

Head of IT Department — FRESH / MoscowFresh / FamilyFriend

07/2018 — 04/2022 | Moscow

Brought back to FRESH after a short stint at Statsbot specifically to lead the technical integration of the company's first acquisition. Stayed on to run the entire in-house technology function during a high-growth, M&A-heavy phase — 2 Moscow warehouses, ~1,000 orders/day.

- Managed a cross-functional team of 10 (backend, frontend, mobile, design, PM); led hiring, mentoring, and technical direction.
- **Led technical integration of acquired competitors** — consolidated their stacks, data, and operations into FRESH systems across multiple M&A events.
- **Architected and built the warehouse management system from scratch** — stock, supplies, shipments, full warehouse-accounting automation — replacing manual operations across both sites.
- **Implemented company-wide business analytics on Metabase**, building on data-analytics experience from Statsbot; gave every department self-serve dashboards on top of operational data.
- Shipped role-specific tools for every department: drivers, logistics managers, cooks, packers, assemblers; integrated the WMS with industrial cash registers and weighing scales over the network (binary protocol — open-source Go driver: [cas](#)).
- Built the customer-facing storefronts (Gatsby / React) and React Native mobile apps; backend on Go + [gqlgen](#) (GraphQL).
- Operated a 30+ VM cluster on Yandex Cloud with Nomad / Consul, Packer / Ansible provisioning, Prometheus / Grafana / AlertManager monitoring, and self-hosted GitLab CI/CD.
- Owned the full PoC → production cycle for every system shipped, often translating vague business requests directly into working tooling.

Stack and tools: Go, gqlgen, GraphQL, React, Gatsby, React Native, TypeScript, PostgreSQL, PostgREST, Postgraphile, Metabase, Nomad, Yandex Cloud, Ansible, Packer, Docker, Prometheus, Grafana, GitLab, Redis.

Fullstack Developer — Statsbot.co

12/2017 — 04/2018 | Moscow

Brief, focused stint at a SaaS data-analytics startup (the codebase later became the open-source project **Cube.js**) between FRESH tenures.

- Shipped features on Ruby on Rails / React in a fast-moving early-stage team.
- **First deep exposure to business analytics and OLAP workflows** — directly led to designing and implementing the company-wide analytics platform after returning to FRESH.

Stack and tools: Ruby, Ruby on Rails, React, Redux, Webpack, Docker.

Fullstack Developer — FRESH

01/2014 — 12/2017 | Moscow

Same company as the later Head of IT role — joined as the sole engineer and grew with the business.

- Took over and stabilized the e-commerce platform (Rails / React) after the previous developer; built the mobile web with SSR.

- Found and fixed a one-line perf bug in [Arbre](#) — the templating core of [ActiveAdmin](#) (~10k stars; the Rails admin framework powering FRESH's backend). ~8x speedup on the affected rendering path. Discovered while maintaining ~100 admin controllers. PR merged upstream.
- Maintained the native iOS app (Objective-C) and migrated it to a React Native rewrite.

Stack and tools: Ruby, Ruby on Rails, React, SSR, React Native, Redux, Webpack, PostgreSQL, Redis, Sidekiq.

Freelance Web Developer

2011 — 2014 | Sevastopol

- Built websites and web apps for SMB clients in PHP / Yii and Ruby on Rails; HTML / CSS / jQuery frontends; PostgreSQL and MongoDB backends.

Stack and tools: PHP, Yii, Ruby, Ruby on Rails, HTML, CSS, jQuery, PostgreSQL, MongoDB.

SKILLS

- **AI / LLM:** OpenAI API, Model Context Protocol (MCP), AI agents, vector indexing, RAG, prompt engineering, agent orchestration, fine-tuning
- **Languages:** Go (Golang)*, TypeScript, Ruby*
- **Backend & Data:** REST, GraphQL, gRPC / Protobuf, BigQuery, PostgreSQL*, SQLite, Redis
- **Cloud & Infra:** Docker*, Kubernetes, Google Cloud, S3, Nomad / Consul, Ansible, Packer
- **Frontend:** React*, Next.js, Gatsby, React Native*, Tailwind

* — Confident with 5+ years of commercial experience

LANGUAGES

- Russian — Native
- Ukrainian — Native
- English — B2+

HOBBIES

Zettelkasten / Second Brain

3+ years on a personal Obsidian graph. Directly seeded [trip2g.com](#).

Electronics

Designing and assembling small devices on popular microcontrollers. Useful skills: C++.

3D Printing

Designing parts in CAD and printing them. Useful skills: CAD proficiency, CNC experience.

Hydroponics

Soil-free plant growth with automation. Useful skills: nutrition / environment monitoring on Prometheus / Grafana.

GameDev

Unity, Godot, VR. Useful skills: C#, C++.